

```

"""
Expression Tree Generator and Evaluator
Python Programming Language (N-PCCCM304P)
Pranjal Parihar - CM25101
"""

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def build_tree(postfix):
    """Builds an expression tree from a
    postfix expression using a stack."""
    stack = []
    for token in postfix:
        node = Node(token)
        if token in '+-*/':
            node.right = stack.pop()
            node.left = stack.pop()
            stack.append(node)
    return stack.pop()

def evaluate(node):
    """Recursively evaluates the expression
    tree."""

```

```

        if node.left is None and node.right is
None:
            return float(node.value)

        left_val = evaluate(node.left)
        right_val = evaluate(node.right)

        if node.value == '+':
            return left_val + right_val
        if node.value == '-':
            return left_val - right_val
        if node.value == '*':
            return left_val * right_val
        if node.value == '/':
            return left_val / right_val

def inorder(node):
    """Recursively traverses the tree in
inorder (left, root, right)."""
    if node is None:
        return []
    return inorder(node.left) +
[node.value] + inorder(node.right)

def postorder(node):
    """Recursively traverses the tree in
postorder (left, right, root)."""
    if node is None:
        return []
    return postorder(node.left) +
postorder(node.right) + [node.value]

```

```
def main():
    expr = input('Enter postfix expression:
').split()
    tree = build_tree(expr)

    print('Inorder traversal :', '
'.join(inorder(tree)))
    print('Postorder traversal :', '
'.join(postorder(tree)))
    print('Result:', evaluate(tree))

if __name__ == '__main__':
    main()
```